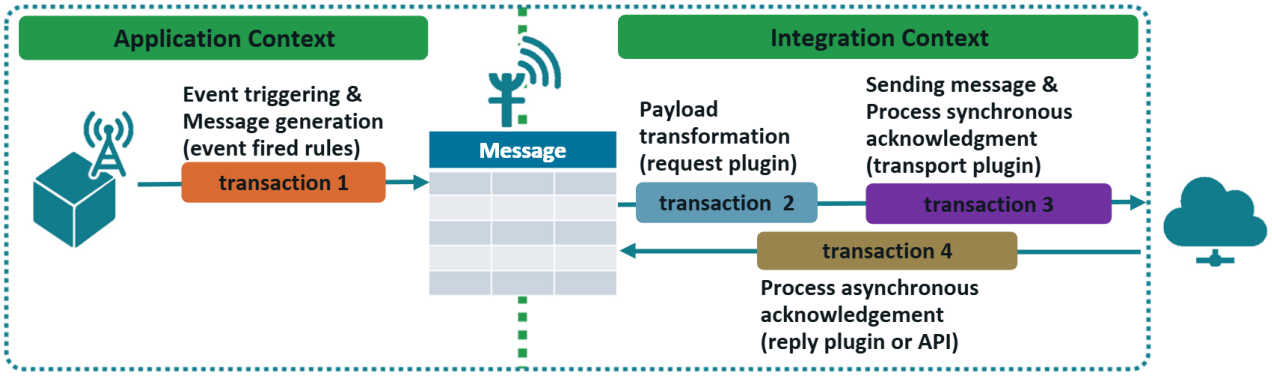
Message Payload Transformation

Guidewire servers and messaging transactions



- All activity from **payload transformation** takes place in **transaction 2**
- Messaging occurs across multiple transactions
  - If an exception is thrown, work in the current transaction is rolled back

A message is always processed in at least two and as many as four transactions:

- Transaction 1 – Creates the message and generates the payload. This transaction always occurs.
- Transaction 2 – Prior to sending the message, an optional transaction may transform the message payload. This is usually done when the message must include information that can only be known immediately before sending the message.
- Transaction 3 – The sending of the message to the external system occurs in its own transaction. This transaction always occurs. If the external system acknowledges the message synchronously, then the acknowledgement is included in this transaction.
- Transaction 4 – If the external system acknowledges the message asynchronously, then a separate transaction processes the acknowledgement. This transaction does not occur for messages that are acknowledged synchronously.

The transaction structure is important because this determines the work that is rolled back if an error occurs.

Notes

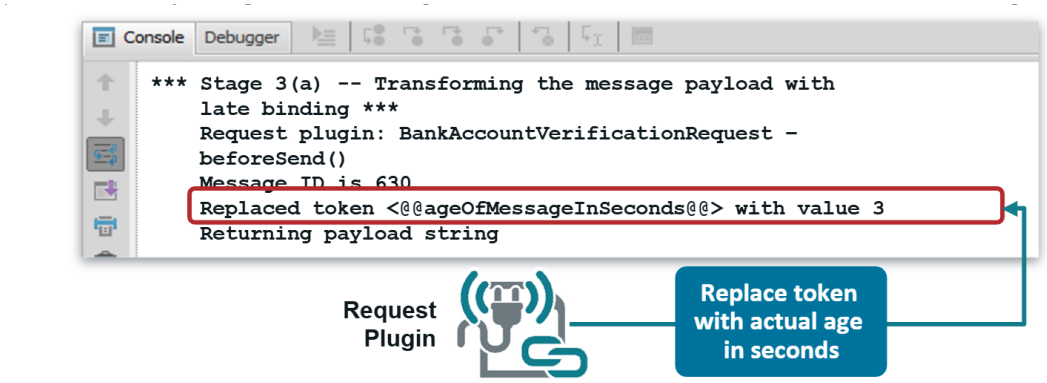
Guidewire servers and messaging transactions

The servers involved at each point of the process are important because they determine where payload work should occur. The majority of payload generation should be done in Event Fired rules on the **ui** role server that manages the given user's session. However, if some portion of payload generation must be done immediately before the message is sent, that portion of the payload can be generated on the **messaging** role server.

Payload transformation

Payload late binding

- A **late binding value** (token) is:
  - A message payload value that cannot be determined when the payload is generated (event-time)
  - Such a value is obtained and substituted in the payload right before the message is sent (send-time)
- The message request plugin:
  - Responsible for replacing all late binding tokens with actual values in an outbound message



Payload late binding

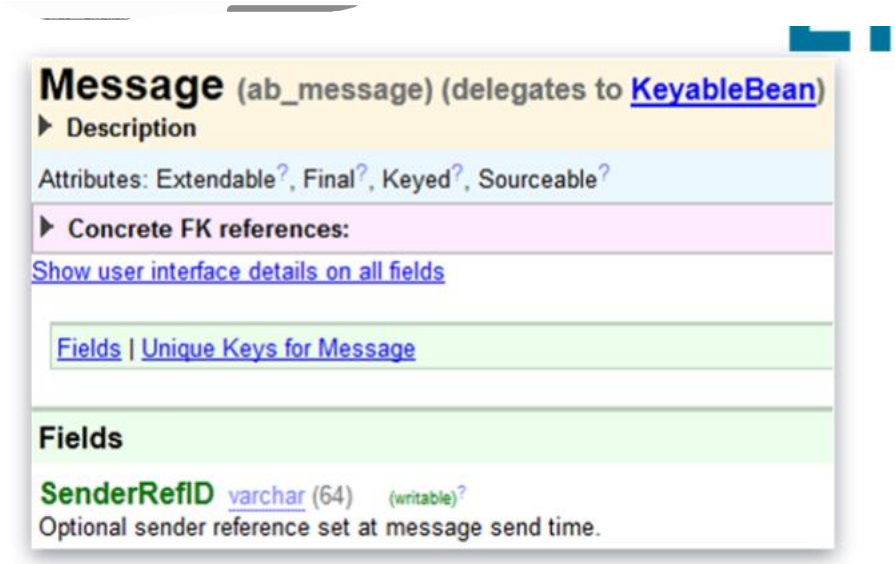
Good candidates for late binding include:
- A send timestamp
- The message ID and/or its public ID
- An identifier from the external system that was sent in reply to a previous message

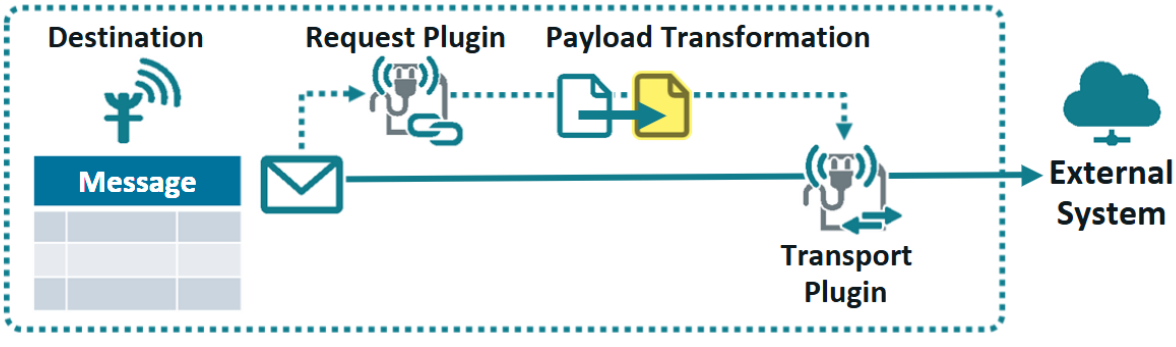
For example, imagine an integration point in which the first message communicates that a new entity was added. The reply to the first message contains the ID to be used for the new entity on the external system. The second message identifies a change on this new entity. However, the second message is created before the first message has been sent (or before the reply to the first message has been received). The second message must use the ID in the reply to the first message. The only way this ID can be added to the second message is through late binding.

Typically, values that are to be late bound are added to the message payload using a placeholder marked by special character delimiters, such as <@@ and @@>. In the message request plugin, these placeholders are replaced with the required data.
Be careful about what data you set via late binding. Typically, the message payload should reflect a consistent snapshot of the system at the time the Event Fired rule executed. Late binding should not be used to capture entity data after message creation.

SenderRefID

- Optional message identifier
  - Determined by Guidewire, but in some cases, the external system determines the SenderRefID
- Used by:
  - The external system to identify which request message it is responding to
  - Guidewire to look up a corresponding pending asynchronous request message
  - System administrator to identify failed messages





Notes

SenderRefID

In some cases, there is no need to set the SenderRefID field:

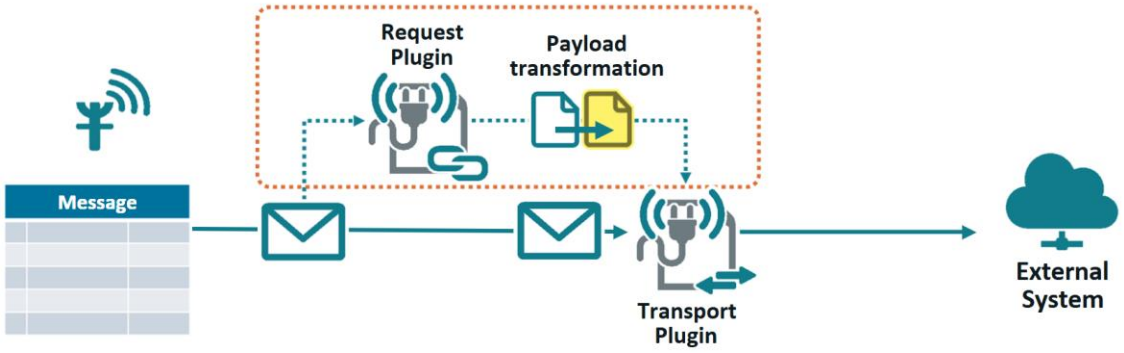
- A message does not need a reply from the external system. These are **fire and finish** messages where Guidewire must simply inform some other system of some event, but Guidewire does not get any response from the other system.
- A message is replied to synchronously at the time that it is sent, and the external system does not need the value.
- In both cases, the application synchronously marks the message as acknowledged in the same code that sends the message.

In some cases, the external system determines the SenderRefID:

- The transport plugin sends the outbound message.
- The transport plugin receives the SenderRefID value from the external system (synchronously).
- The transport plugin sets the SenderRefID field on the message.

Request plugin

Message request plugin



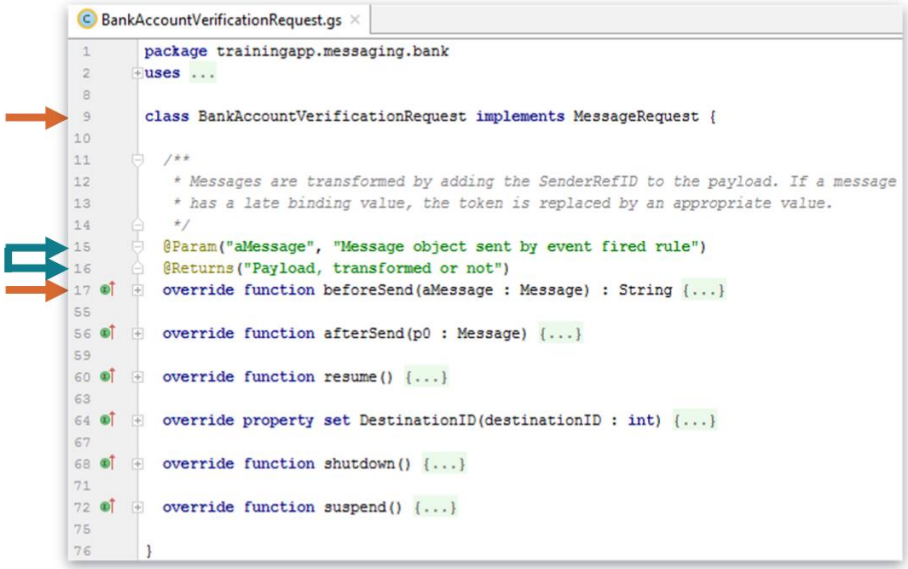
- Called before the payload is sent by transport plugin
- Sets the SenderRefID on the message object

- Transforms the payload
  - Sets late binding value(s)
  - Optionally, adds the SenderRefID value to the message payload

Create a message request plugin class

Use a package naming convention recommendation:

- **<company>.<app_code>.messaging.<functional_area>**



Notes

Create a message request plugin class

Code example
Line 9
- Implement the MessageRequest interface.
- Message entity instance is in transaction 2 scope.

Lines 15-16
- Method annotations added.

Line 17
- Must override beforeSend method.
- Message object is passed.
- Payload transformation is done in the request plugin's beforeSend method.
- It is called immediately before the message is sent.
- The method must always return a transformed payload. If the payload is not transformed, then the original payload should be returned.

afterSend method
Performs post-processing on the Message object immediately after sending the message with the send method of the MessageTransport plugin. Typically, not implemented.

resume, suspend, and shutdown methods
- Are called by the application when the destination is resumed, suspended, or the server shuts down. Plugins should acquire or release external resources at these times. For example, a plugin may need to create or close JMS or JDBC connections, JMS Sessions, sockets, and so on.

and so on.
- Typically, these methods are empty in a request plugin. In a production environment, they should be implemented for transport and reply plugins.
- The plugin should throw a Guidewire InitializationException (gw.plugin.messaging.InitializationException) from resume if the resources have not initialized successfully. (A manual resume could be attempted later and may be successful.)

set DestinationID property
- Is not related to how a message is suspended or resumed.
- It allows your destination to get its own destination ID to store it in a private variable. Your code can use the stored destination ID value for logging messages or send it to integration systems so that they can programmatically suspend/resume the destination if necessary. Keep in mind that the valid range for your destination IDs is 0 through 63, inclusive.
- Guidewire reserves all other destination IDs for built-in destinations, such as the email transport destination.

Set the SenderRefID

```
17  override function beforeSend(aMessage : Message) : String {
18    // Set the SenderRefID
19    if (aMessage.SenderRefID == null) {
20      aMessage.SenderRefID = aMessage.PublicID
21    }
```

- You must ensure that your `beforeSend` method implementation is **idempotent**, which means that it could be called multiple times and have the same effect.
- Set the SenderRefID only once for a new message

Notes

Set the SenderRefID

Code example
Line 19
- Determine if the SenderRefID hasn't already been set for this message.
- A single SenderRefID should be used for all attempts to send the same message.
- Application may need to attempt to send multiple times if it encounters errors.

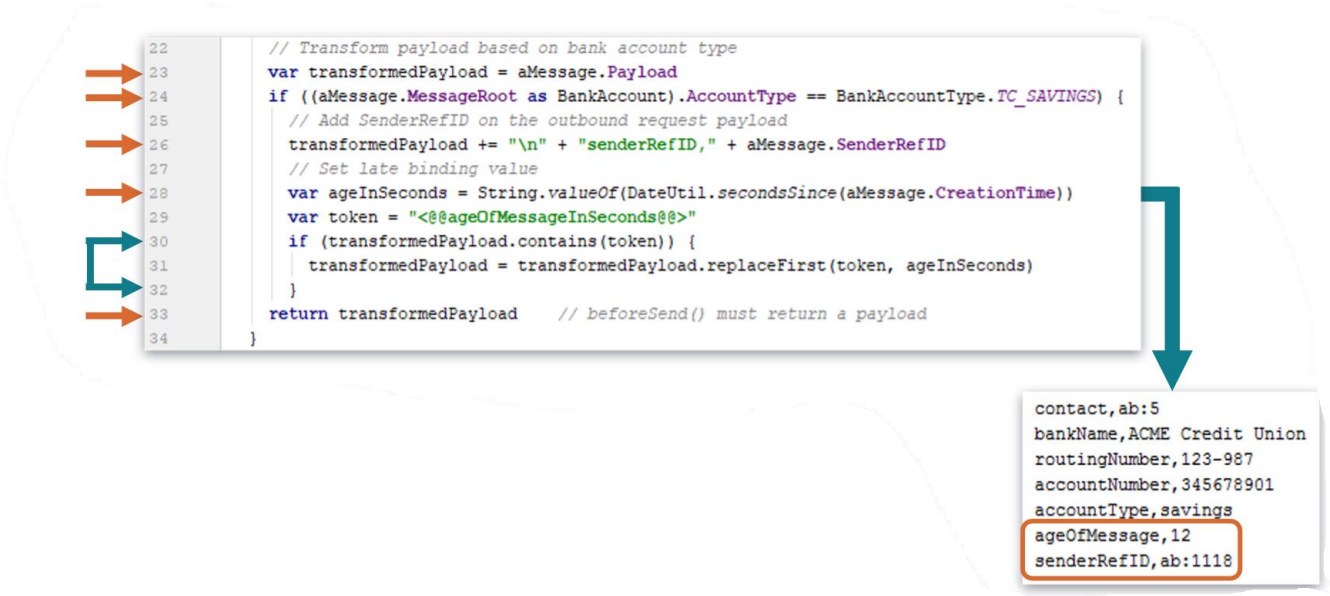
Line 20
- Set the SenderRefID in the massage typically using the PublicID field.

When an asynchronous reply is received, the SenderRefID is used to locate the message to acknowledge, even if the message was sent multiple times due to error. As long as all automatically retried messages have the same SenderRefID as the original, Guidewire can find the correct message to acknowledge, regardless of which specific instance the external system is replying to.

PublicID
All entities, including Message, have a PublicID field. The purpose of this field is to identify instances of the entity as they are known by the external system. Thus, the purpose of the SenderRefID field is identical to that of the PublicID field. The Message entity has two public identification fields because earlier versions of the Guidewire applications did not have the PublicID field across all entities. However, the Message entity absolutely required a settable field that could be shared with external systems, and so Message was created with the SenderRefID field. As the Guidewire applications matured, the PublicID field was added to entities in the data model. This allowed other entities beyond Message to be shared with external systems.

Even though you theoretically could use Message's PublicID field, Guidewire recommends you use SenderRefID instead. There are numerous base application methods that make use of SenderRefID, such as IMessagingToolsAPI's getMessageID method that takes the SenderRefID as an input and the MessageFinder's findHistoryBySenderRefID method. Using PublicID to identify a message could make an implementation more complex.

Transform the payload



- The `beforeSend` method should **not** modify the message payload directly
- It should return a transformed payload

Notes

Transform the payload

Code example
Line 23
- Create the transformedPayload variable based on the original payload.

Line 24
- Determine if the message is based on a savings account.

Line 26
- Transform the payload by adding the SenderRefID.

Line 28
- Calculate the age of the message by comparing the time the message was created to the time just before send.

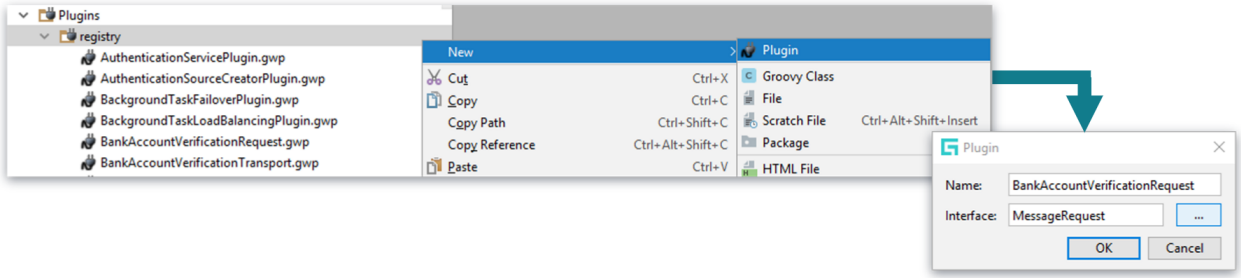
Lines 30-32
- Transform the payload by replacing the token with the calculated ageInSeconds variable.

Line 33
- Must return a payload that will be passed to the send method in the transport plugin.

You can modify the message payload using a placeholder or token. Often, tokens and placeholders are designated with special character delimiters, such as <@@ and @@>. In the beforeSend method, these placeholders are replaced with the required data for late binding.

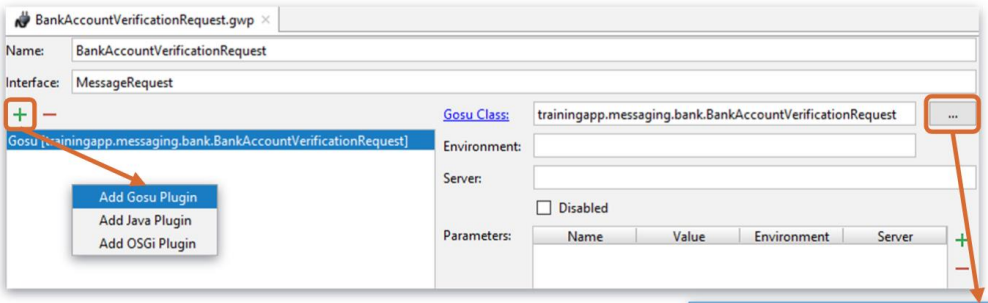
In some situations, the initial sending of the message is unsuccessful. This could be because of a problem with the Guidewire application or with the external system. In these situations, the message needs to be resent. Because a message may be sent multiple times, the beforeSend method may be called multiple times. Therefore, the beforeSend method should **not** modify the message payload directly. Instead, it should return a transformed payload. This lets later attempts to send the message transform the payload as appropriate.

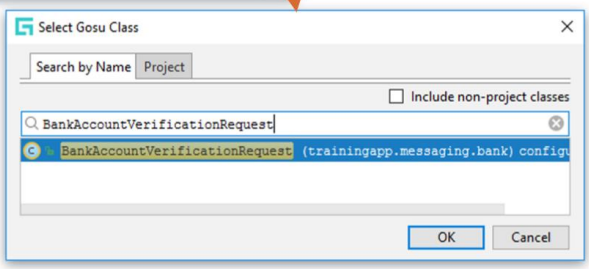
Create plugin registry



- **configuration.config.Plugins.registry**
  - Specify a new request plugin registry entry name
  - Specify the interface to be implemented
- Add the type of plugin to be implemented in the plugin name

Configure the plugin



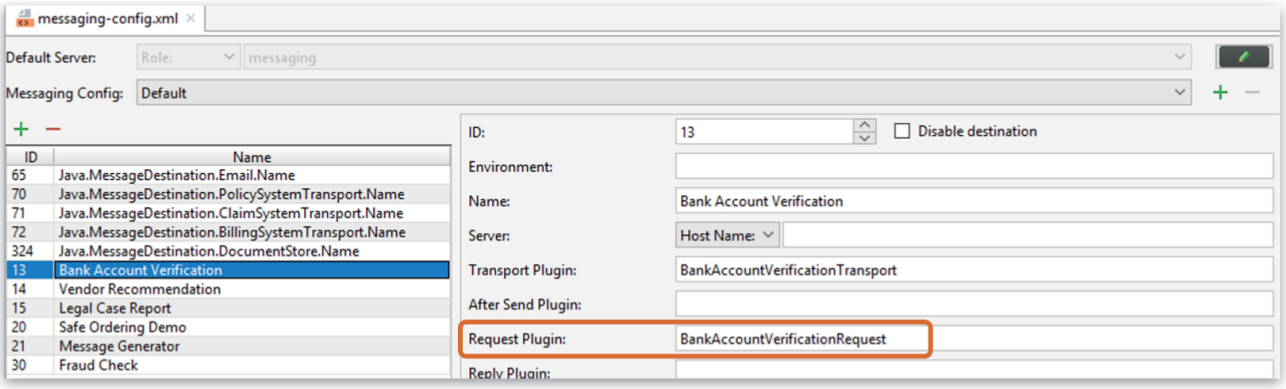


- Add Gosu Plugin
  - Optionally, select Java or OSGi plugin
- Click the ellipse (...) and select the plugin implementation class in the dialog

Configure the plugin

This plugin does not implement parameters.

Configure the destination



- Navigate to:
  - **configuration.config.Messaging.messaging-config.xml**
- The destination must point to a plugin registry element that references the appropriate Message Request plugin

Notes

Configure the destination

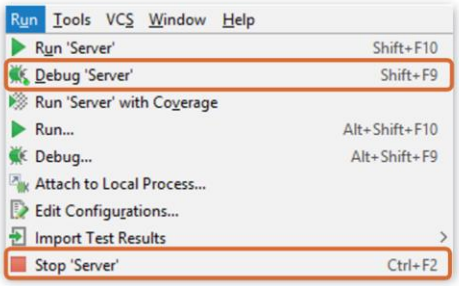
For pre-defined plugins (such as authentication), you can implement only one version of each plugin at a time. For messaging plugins, you can implement any number of MessageRequest, MessageTransport, and MessageReply plugins.

Every destination can use only one message request plugin.

Deploy Code Changes

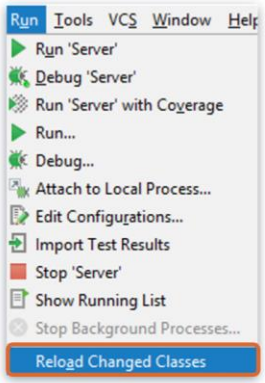
Restart Server

- New request plugin class
- New or modified plugin registry
- New or modified destination



Reload Changed Classes

- Modified request plugin class



Notes

Deploy code changes

You can restart the server from Studio by stopping a running server and after the server stops, then starting the server again in either Run or Debug mode (preferred for training).

Question and Answer

With regards to payload generation and payload transformation:

A) On which server does each take place?

B) At what point in time does each take place?

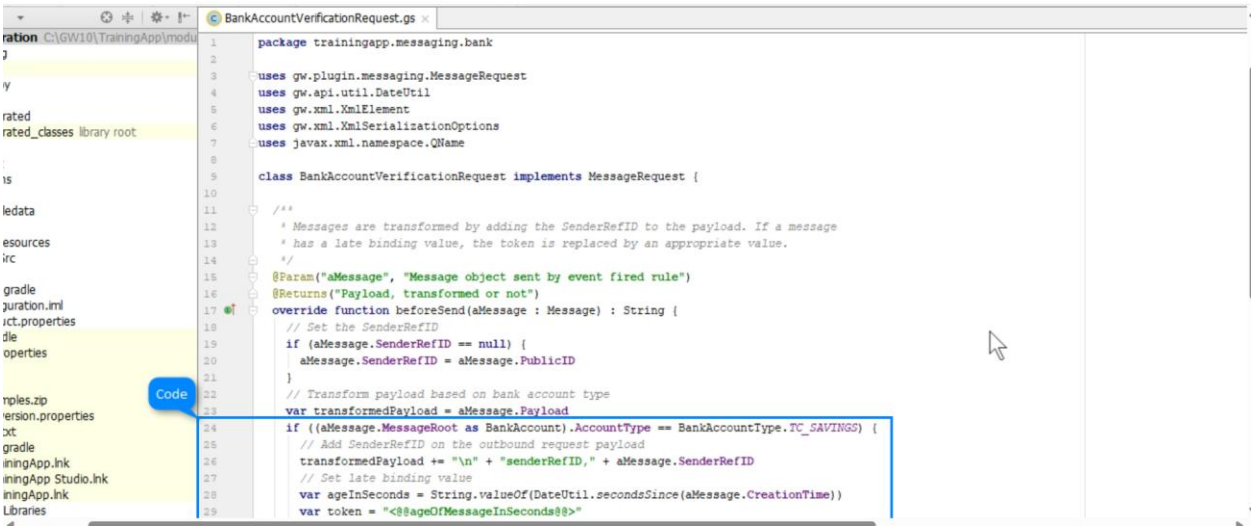
A) Payload generation occurs on the application/UI server. Payload transformation occurs on the batch/integration server.

B) Payload generation occurs during Event Fired rules (right after the triggering event occurs). Payload transformation occurs right before the message is sent (in the request plugin's beforeSend method.

Under what circumstance would you need to late bind a message payload?

You would need to late bind a message if information that needs to go in the message payload is not known at the time the message payload is generated in Event Fired rules.

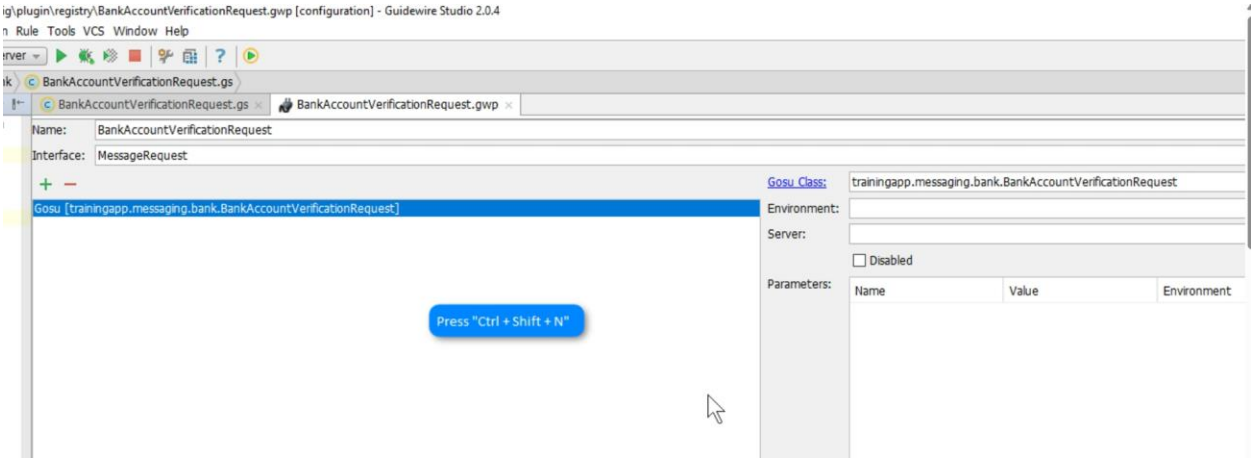
Demonstration

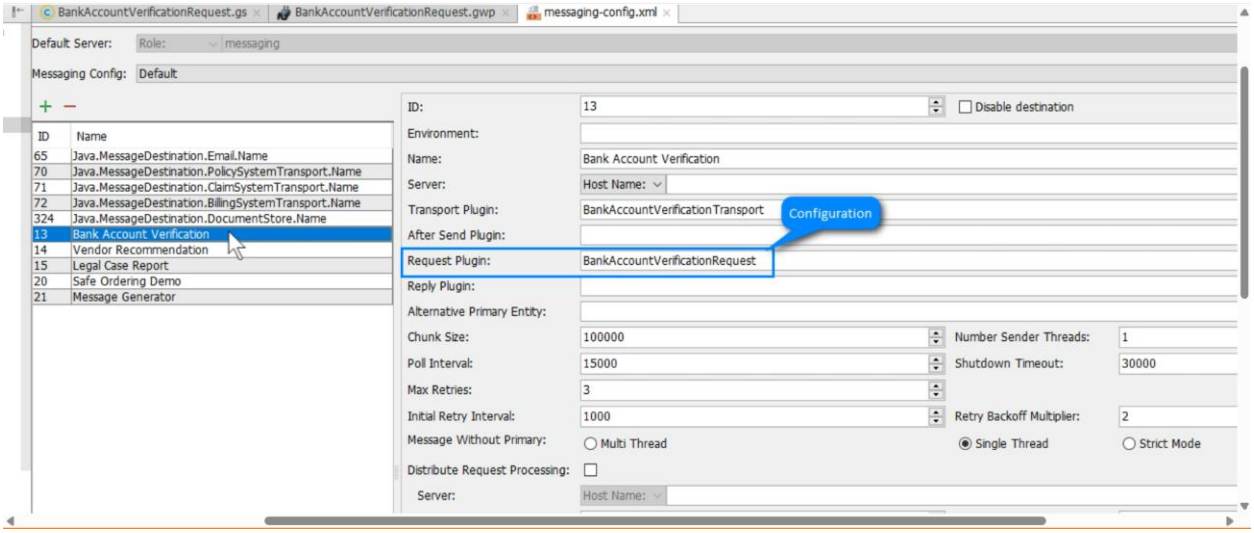


```
var transformedPayload = aMessage.Payload
if ((aMessage.MessageRoot as BankAccount).AccountType == BankAccountType.TC_SAVINGS) {
  // Add SenderRefID on the outbound request payload
  transformedPayload += "\n" + "senderRefID," + aMessage.SenderRefID
  // Set late binding value
  var ageInSeconds = String.valueOf(DateUtil.secondsSince(aMessage.CreationTime))
  var token = "<@@ageOfMessageInSeconds@@>"
  if (transformedPayload.containsIgnoreCase(token)) {
    transformedPayload = transformedPayload.replace(token, ageInSeconds)
  }
} else {
    // Add SenderRefID on the outbound request payload
    var xml = XmlElement.parse(transformedPayload)
    var namespace = xml.$Namespace.NamespaceURI
    var senderRefID = new XmlElement(new QName(namespace,"SenderRefID"))
    senderRefID.set$Text(aMessage.SenderRefID)
    xml.addChild(senderRefID)
    // Create transformedPayload with serialization options
    var opts = new XmlSerializationOptions()
    opts.XmlDeclaration = false
    opts.Sort = false
    opts.Validate = false
    transformedPayload = xml.asUTFString(opts)
}
// Print statement is for training purposes only
print("\n*** Stage 3 -- Optionally transform the message payload ***" +
      "\n    Request plugin: BankAccountVerificationRequest -- beforeSend()" +
      "\n    Sending transformed payload to console:" +
      "\n" + transformedPayload)
```

```
  // Print statement is for training purposes only
  print("\n*** Stage 3 -- Optionally transform the message payload ***" +
        "\n    Request plugin: BankAccountVerificationRequest -- beforeSend()" +
        "\n    Sending transformed payload to console:" +
        "\n" + transformedPayload)

  return transformedPayload    // beforeSend() must return a payload
}
```


ig\plugin\registry\BankAccountVerificationRequest.gwp [configuration] - Guidewire Studio 2.0.4
n Rule Tools VCS Window Help
BankAccountVerificationRequest.gs
BankAccountVerificationRequest.gs
BankAccountVerificationRequest.gwp
Name: BankAccountVerificationRequest
Interface: MessageRequest
Gosu [trainingapp.messaging.bank.BankAccountVerificationRequest]
Press "Ctrl + Shift + N"
Gosu Class: trainingapp.messaging.bank.BankAccountVerificationRequest
Environment:
Server:
Disabled
Parameters:
Name
Value
Environment


BankAccountVerificationRequest.gs
BankAccountVerificationRequest.gwp
messaging-config.xml
Default Server: Role: messaging
Messaging Config: Default
ID Name
65 Java.MessageDestination.Email.Name
70 Java.MessageDestination.PolicySystemTransport.Name
71 Java.MessageDestination.ClaimSystemTransport.Name
72 Java.MessageDestination.BillingSystemTransport.Name
324 Java.MessageDestination.DocumentStore.Name
13 Bank Account Verification
14 Vendor Recommendation
15 Legal Case Report
20 Safe Ordering Demo
21 Message Generator
ID: 13
Disable destination
Environment:
Name: Bank Account Verification
Server: Host Name:
Transport Plugin: BankAccountVerificationTransport
Configuration
After Send Plugin:
Request Plugin: BankAccountVerificationRequest
Reply Plugin:
Alternative Primary Entity:
Chunk Size: 100000
Number Sender Threads: 1
Poll Interval: 15000
Shutdown Timeout: 30000
Max Retries: 3
Initial Retry Interval: 1000
Retry Backoff Multiplier: 2
Message Without Primary:
Multi Thread
Single Thread
Strict Mode
Distribute Request Processing:
Server: Host Name:


Messaging Destination ID is 13
Event Fired rules: BAEF1100 - Bank Account
EventAware entity is BankAccount
Event Fired rules: BAEF1110 - Added Changed
Event is BankAccountAdded
*** Stage 2 -- Creating the message ***
Event Fired rules: BAEF1111 - Field Changed
Sending payload to console:
contact,ab:20685
bankName,Succeed Credit Union
routingNumber,001-234
accountNumber,3445500912
accountType,savings
ageOfMessage,<@@ageOfMessageInSeconds@@>
*** Stage 3 -- Optionally transform the message payload ***
Request plugin: BankAccountVerificationRequest -- beforeSend()
Sending transformed payload to console:
contact,ab:20685
bankName,Succeed Credit Union
routingNumber,001-234
accountNumber,3445500912
accountType,savings
ageOfMessage,3
senderRefID,ab:304
Output
rchiriboga-p sys f95f09c8-7d0f-4e16-b0c5-9d31c1bd991f 2019-02-01 18:16:06,354 INFO <MessageBeforeSend> Message payload transformed before sending {messageId="304",
rchiriboga-p sys f95f09c8-7d0f-4e16-b0c5-9d31c1bd991f 2019-02-01 18:16:06,385 INFO <WebServiceRequest> WS request processed {service="{http://example.com/external
rchiriboga-p sys f95f09c8-7d0f-4e16-b0c5-9d31c1bd991f 2019-02-01 18:16:06,386 INFO <MessageAckReceived> Message ack received {messageId="304", currentTime=1549062
*** Stage 4 -- Sending the message ***